



ACS 311 –Principles of Programming Languages

Course: ACS 311 – Principles of Programming Languages

Project Duration: 4 Weeks

Group Size: 4–5 Students

Total Marks: 100 (+10 Bonus Marks)

Programming Language: Any programming language of the group's choice (e.g., C, C++, Java, Python, C#, JavaScript, Go, Rust, Kotlin, Swift, PHP)

Project Overview

This group project is designed to assess your understanding of the **fundamental principles of programming languages** through the design and implementation of a real-world software application.

Your application should demonstrate not only that it solves a practical problem but also that it applies the programming language concepts studied throughout the course, including syntax, variables, data types, control structures, modular programming, abstraction, object-oriented programming (where applicable), file handling, and error handling.

Groups are free to choose any programming language. However, during the project presentation, each group must justify why the chosen language was appropriate for the problem being solved.

Learning Outcomes

Upon successful completion of this project, students should be able to:

- Apply programming language syntax correctly.
- Design modular software using functions, procedures, or methods.
- Demonstrate appropriate use of variables and data types.
- Apply expressions, operators, and control structures to solve problems.
- Implement arrays, lists, or equivalent collections.
- Apply object-oriented programming principles where supported.
- Develop reusable and maintainable code.
- Handle errors and exceptions appropriately.
- Implement persistent data storage using files or databases.
- Test, document, and present a complete software solution.

Project Scenarios (Choose ONE)

Groups should select **one** of the following project scenarios.

1. University Student Information System

Develop a system to manage student records, course registration, grades, transcripts, and academic reports.

2. Smart Hospital Management System

Manage patients, doctors, appointments, prescriptions, medical records, and billing.

3. Library Management System

Manage books, borrowing, returns, reservations, overdue fines, and member records.

4. Restaurant Ordering and Billing System

Allow customers to place orders, generate bills, manage menus, and track sales.

5. Hotel Reservation System

Manage room bookings, guest check-in/check-out, room availability, and invoices.

6. Supermarket Point of Sale (POS)

Handle product inventory, customer purchases, receipts, discounts, and sales reports.

7. Online Examination System

Create quizzes, manage students, automatically mark examinations, and generate reports.

8. Banking Management System

Support account creation, deposits, withdrawals, transfers, balance enquiries, and transaction history.

9. Vehicle Rental System

Manage customers, vehicle bookings, rental charges, returns, and maintenance schedules.

10. School Timetable Generator

Automatically allocate lecturers, classrooms, and time slots while detecting scheduling conflicts.

11. Event Management System

Manage event registration, attendance, ticket generation, and participant reporting.

12. Smart Parking Management System

Track vehicle entry and exit, allocate parking spaces, calculate parking fees, and produce reports.

13. Personal Finance and Budget Manager

Track income, expenses, savings goals, budgets, and monthly financial summaries.

14. Employee Payroll Management System

Manage employee information, calculate salaries, deductions, taxes, and generate payslips.

15. Airline Reservation System

Manage flights, passenger bookings, seat allocation, ticket generation, and cancellations.

16. Pharmacy Inventory System

Manage medicines, suppliers, expiry dates, prescriptions, and stock levels.

17. Clinic Management System

Maintain patient records, doctor appointments, prescriptions, billing, and treatment history.

18. Electronic Voting System

Register voters and candidates, securely cast votes, count results, and generate election reports.

19. Project and Task Management System

Create projects, assign tasks, manage deadlines, and monitor project progress.

20. E-Commerce Store

Manage products, customers, shopping carts, orders, payments, and inventory.

Minimum Functional Requirements

Every project must include:

- A menu-driven, graphical, or web-based user interface.
- Create, Read, Update, and Delete (CRUD) functionality.
- Input validation.
- Appropriate error and exception handling.
- Persistent data storage using files or a database.
- Search functionality.

- Reports or summaries.
- Modular program structure.

Programming Language Concepts to Demonstrate

Each project will be assessed on how effectively it demonstrates the following programming language concepts.

Programming Language Concept	Marks
Variables and Data Types	5
Expressions and Operators	5
Selection Statements (Decision Making)	5
Iteration (Loops)	5
Functions, Procedures, or Methods	10
Arrays, Lists, or Other Collections	10
Classes and Objects or Equivalent Abstraction Mechanisms	10
Modular Programming and Code Organization	10
Exception and Error Handling	10
File Handling or Database Integration	10
User Interface (CLI, GUI, or Web)	10
Code Readability, Documentation, and Maintainability	10

Total: 100 Marks

Project Timeline

Week 1 – Planning and System Design (25 Marks)

Each group should:

- Form groups of 4–5 members.
- Select one project scenario.
- Define the problem statement.
- Develop project objectives.
- Identify functional and non-functional requirements.
- Develop algorithms using pseudocode.
- Create a flowchart.
- Prepare a Use Case Diagram.
- Develop a Class Diagram (where applicable).
- Design user interface mock-ups.
- Assign responsibilities among group members.

Deliverables

- Project proposal
- Flowchart
- Pseudocode
- Use Case Diagram
- Class Diagram (if applicable)
- User Interface Design

Week 2 – Core Development (25 Marks)

Develop the main functionality of the application, demonstrating:

- Variables and data types.
- Expressions and operators.
- Selection statements.
- Iteration.
- Functions and procedures.
- Classes and objects (where applicable).
- Data structures.
- Modular programming.
- User interface implementation.

Deliverables

- Working prototype demonstrating the core features of the application.

Week 3 – Testing, Integration, and Refinement (25 Marks)

Complete the application by implementing:

- Input validation.
- Exception handling.
- File handling or database integration.
- Search functionality.
- Reports and summaries.
- Code optimization and refactoring.
- Functional testing.
- Bug fixing.

Deliverables

- Fully functional application ready for presentation.

Week 4 – Final Submission and Demonstration (25 Marks)

Each group must submit:

- Complete source code.
- Executable application (where applicable).
- Technical Report (8–10 pages).
- User Manual.
- Presentation slides.

Each group will deliver a **15-minute presentation** comprising:

- **10 minutes** for the system demonstration.
- **5 minutes** for questions and answers.

Every group member must actively participate in the presentation and explain the component they implemented.

Technical Report Structure

The report should include:

1. Cover Page
2. Group Members
3. Introduction
4. Problem Statement
5. Objectives
6. Functional Requirements
7. Non-functional Requirements
8. System Design
9. Justification for the Chosen Programming Language
10. System Implementation
11. Testing and Results
12. Challenges Encountered
13. Lessons Learned
14. Future Improvements
15. References

Assessment Rubric

Assessment Area	Marks
Week 1: Planning and System Design	25
Week 2: Core Development	25
Week 3: Testing, Integration, and Refinement	25
Week 4: Final Submission and Demonstration	25
Total	100

Bonus Marks – GitHub Submission (Optional)

GitHub is **not a compulsory requirement** for this project. However, groups that use GitHub effectively to manage and document their project development may earn **up to 10 bonus marks**.

Maximum Bonus: 10 Marks

Recommended GitHub Repository Structure (Optional)

```
ProjectName/  
├── README.md  
├── src/  
├── docs/  
├── screenshots/  
├── presentation/  
└── LICENSE (optional)
```

Academic Integrity

- Each group's solution must be **original**.
- Any external libraries, frameworks, or third-party resources must be properly acknowledged.
- AI tools (e.g., ChatGPT, GitHub Copilot, Gemini, Claude) may be used to support learning, debugging, or documentation. However, every group member must understand and be able to explain the code they contributed.
- During the project demonstration, any group member may be asked to explain any part of the system. Failure to adequately explain the implementation may result in reduced marks for the individual or the group.
- Plagiarism, copied projects, or unauthorized collaboration between groups will be treated as academic misconduct in accordance with the University's academic integrity policy.

Submission Checklist

Each group must submit the following before the deadline:

-  Complete source code
-  Executable application (where applicable)
-  Technical Report (8–10 pages)
-  User Manual
-  Presentation slides
-  Demonstration during the scheduled presentation session
-  Optional GitHub repository for bonus marks

Note: The emphasis of this project is on demonstrating the **principles of programming languages** through the design of a well-structured, modular, maintainable, and functional software application, rather than the complexity of the chosen technology stack.

